CYBORG SUPERMARKET ENTERPRISE DOCUMENTATION

System Architecture and Infrastructure

Clean Architecture, Onion Layers, and Infrastructure Topology

DOCUMENT ID: 02_System_Architecture_and_Infrastructure.pdf

Executive Publication Record

Organization:	Cyborg Omnichannel Engineering Organization
Ecosystem:	Windows POS • Android Mobile • Web Storefront • Admin Web • Backend
Primary Authority:	cyborg_system_architecture.pdf (16-Page Blueprint)
Source Dataset:	super_market(1).xlsx (1,000 Verified Egyptian Retail Records)
Security Level:	Enterprise Retail Distribution / Stakeholder Review
System Stack:	NestJS • TypeScript • PostgreSQL 16 • Redis 7 • SQLite • Flutter
Target Currency:	Egyptian Pounds (EGP) • Cairo Timezone (UTC+2 / UTC+3)

1. Clean Architecture (Onion) Decomposition

The Cyborg backend strictly complies with Onion Architecture principles to guarantee that enterprise domain rules remain isolated from databases, HTTP frameworks, and external third-party services.

Layer Breakdown & Isolation Rules

- Layer 1 - Domain Entities: Pure business models (Product, Order, Inventory, Shift). No external framework imports.

- Layer 2 - Application Use Cases: Orchestrates business workflows (CreateOrder, SyncInventory, ProcessVoiceCommand).

- Layer 3 - Interface Adapters: Controllers, WebSocket gateways, Presenters, and DTO serializers.

- Layer 4 - Infrastructure & Frameworks: PostgreSQL TypeORM drivers, Redis pub/sub adapters, and SQLite engines.

2. Physical Network Topology & Containerization

The ecosystem deploys locally via Docker Compose and scales in production using managed PostgreSQL (RDS) and Redis clusters:

Service Name	Container Image	Exposed Port	Storage Mount / Role
backend-api	node:18-alpine (NestJS)	Port 3000 (HTTP/WS)	Stateless API server with auto
postgres-db	postgres:16-alpine	Port 5432 (TCP)	Persistent pgdata volume + pgv
redis-bus	redis:7-alpine	Port 6379 (TCP)	In-memory Pub/Sub event bus &
admin-web	node:18-alpine (Next.js)	Port 3001 (HTTP)	Responsive operations dashboar
customer-web	node:18-alpine (Next.js)	Port 3002 (HTTP)	Customer-facing digital storef

3. POS Terminal Architecture & Offline Durability

The desktop POS terminal runs as a compiled Windows native application with an embedded SQLite engine. When connectivity is interrupted, sales continue without degradation:

- Local Durability: Every completed transaction is written to SQLite before printing receipt.

- Outbox Table: An asynchronous worker queues pending records in sync_outbox with retry policies.

- Reconnection Delta Sync: Upon network recovery, the terminal streams transactions to /api/v1/sync/ingest.